Low-Level Design (LLD) — RAG Customer Support Assistant

Field	Value
Project	RAG-Based Customer Support Assistant with LangGraph & HITL
Role	Senior AI Engineer
Date	April 2026
Version	1.0

---

1. Module-Level Design

1.1 Document Processing Module (`src/ingestion.py`)

Purpose: Load PDF files and extract text content on a per-page basis, producing LangChain `Document` objects that preserve source metadata for downstream attribution.

Input: File path (string) to a `.pdf` file, or directory path containing multiple PDFs.

Output: `List[Document]` — each `Document` has `.page_content` (string) and `.metadata` (dict with `source` and `page` keys).

Internal Logic:

1. Validate file/directory existence; raise `FileNotFoundError` if missing.
2. For directories, use `glob.glob("*.pdf")` to discover all PDFs.
3. Instantiate `PyPDFLoader(file_path)` per file and call `.load()`.
4. Concatenate all page-level Documents into a single list.
5. Pass the combined list to the chunking function.

1.2 Chunking Module (`src/ingestion.py`)

Purpose: Split page-level Documents into smaller, retrieval-optimized chunks that fit within embedding model context windows and contain focused semantic content.

Input: `List[Document]` from the document loader; optional `chunk_size` (int, default 500) and `chunk_overlap` (int, default 50).

Output: `List[Document]` — each chunk inherits parent metadata and has `page_content` of at most `chunk_size` characters.

Internal Logic:

1. Initialize `RecursiveCharacterTextSplitter` with the given `chunk_size`, `chunk_overlap`, and separator hierarchy: `["\n\n", "\n", ". ", " ", ""]`.
2. Call `splitter.split_documents(documents)` which recursively tries each separator starting from the most meaningful boundary (double newline = paragraph break) and falls back to less meaningful boundaries.
3. Each resulting chunk preserves the source document's metadata.
4. Print diagnostic: total chunks created, preview of first chunk.

1.3 Embedding Module (`src/vector_store.py`)

Purpose: Convert text strings into 384-dimensional dense vectors suitable for cosine similarity search.

Input: Text string (single chunk or query).

Output: `List[float]` — 384-dimensional L2-normalized vector.

Internal Logic:

1. Load `all-MiniLM-L6-v2` via `HuggingFaceEmbeddings` with explicit `device="cpu"` and `normalize_embeddings=True`.
2. The model tokenizes input text, passes it through a 6-layer Transformer encoder, and produces a sentence-level embedding via mean pooling.
3. L2 normalization ensures cosine similarity reduces to a dot product, simplifying downstream computation.
4. The embedding function is initialized once and reused across all embed calls within a session.

1.4 Vector Storage Module (`src/vector_store.py`)

Purpose: Persist embedded chunks to disk and provide indexed similarity search.

Input: `List[Document]` (chunks) for storage; query string for retrieval.

Output: For storage: `Chroma` instance. For retrieval: `(List[Document], List[float])` — documents and scores.

Internal Logic:

1. **Create**: `Chroma.from_documents()` embeds all chunks in batch and stores vectors + metadata + text in a SQLite-backed collection at `CHROMA_PERSIST_DIR`.
2. **Load**: `Chroma()` constructor with `persist_directory` loads the existing collection without re-embedding.
3. **Search**: `similarity_search_with_relevance_scores(query, k=top_k)` embeds the query, performs HNSW approximate nearest neighbor search, and returns `(Document, score)` tuples sorted

by descending relevance.

1.5 Retrieval Module (`src/vector_store.py::retrieve_relevant_chunks`)

Purpose: Execute a similarity search and return the top-k results with relevance scores for downstream confidence computation.

Input: `Chroma` vector store instance, query string, `top_k` integer.

Output: Tuple of `(List[Document], List[float])` — matched documents and their relevance scores (0-1, higher = more relevant).

Internal Logic:

1. Call `vector_store.similarity_search_with_relevance_scores(query, k=top_k)`.
2. Unpack results into separate document and score lists.
3. Print a preview of each retrieved chunk with its score for demo visibility.

1.6 Query Processing Module (`src/graph.py::generate_node`)

Purpose: Construct a grounded prompt from retrieved chunks and generate an LLM response.

Input: `SupportState` containing `query` and `retrieved_chunks`.

Output: Updated state with `response` field populated.

Internal Logic:

1. Format retrieved chunks into a numbered context block: `[Chunk 1]: ...`, `[Chunk 2]: ...`, etc.
2. Construct a `ChatPromptTemplate` with a system message that instructs the LLM to answer only from context and flag uncertainty with "ESCALATE".
3. Chain: `prompt | llm | StrOutputParser()`.
4. Invoke the chain with `{"context": context, "query": query}`.
5. Write the response string to state.

1.7 Graph Execution Module (`src/graph.py`)

Purpose: Define and compile the LangGraph StateGraph that orchestrates the RAG pipeline.

Input: `SupportState` with `query` field populated.

Output: Final `SupportState` with all fields populated.

Internal Logic: See Section 4 (LangGraph Workflow Design) below.

1.8 HITL Module (`src/hitl.py`)

Purpose: Determine whether a query requires human intervention and, if so, collect a human response via terminal input.

Input: `confidence` (float), `response` (string) for escalation check; `query`, `llm_response`, `confidence` for human collection.

Output: `bool` for escalation check; `str` for human response.

Internal Logic: See Section 6 (HITL Design) below.

---

2. Data Structures

2.1 SupportState (Graph State)

```
from typing import TypedDict, List, Optional

class SupportState(TypedDict):
    query: str                      # Customer's natural language question
    retrieved_chunks: List[str]     # Text content of top-k retrieved chunks
    confidence: float               # Average retrieval similarity score [0, 1]
    response: str                   # Final response (LLM or human)
    escalate: bool                  # Whether HITL was triggered
    human_response: Optional[str]   # Human operator's response (None if not
escalated)
```

Design rationale: `TypedDict` was chosen over `dataclass` because LangGraph's `StateGraph` requires a dict-like state type for its internal merge semantics. Each node returns a partial dict that gets merged into the current state — `TypedDict` provides type checking without introducing class instantiation overhead.

2.2 Chunk Format (Document + Metadata)

```
# LangChain Document structure (from langchain_core.documents)
{
    "page_content": "Our return policy allows customers to return...",
    "metadata": {
        "source": "data/sample_support_docs.pdf",
        "page": 0
    }
}
```

After chunking, each chunk retains the parent document's metadata, enabling source attribution in the response.

2.3 Embedding Structure

```
# 384-dimensional L2-normalized vector
embedding: List[float]  # len = 384, ||embedding||_2 = 1.0

# ChromaDB internal storage per chunk:
{
    "id": "uuid-auto-generated",
    "embedding": [0.0123, -0.0456, ...],   # 384 floats
    "document": "Our return policy allows...",
    "metadata": {"source": "...", "page": 0}
}
```

2.4 Query-Response Schema

```
# Input to the graph
input_state = {
    "query": "What is your return policy?",
    "retrieved_chunks": [],
    "confidence": 0.0,
    "response": "",
    "escalate": False,
    "human_response": None
}

# Output from the graph (normal path)
output_state = {
    "query": "What is your return policy?",
    "retrieved_chunks": ["Our return policy allows...", "Items must be...",
"Refunds are..."],
    "confidence": 0.72,
    "response": "Our return policy allows returns within 30 days...",
    "escalate": False,
    "human_response": None
}

# Output from the graph (HITL path)
output_state_hitl = {
    "query": "I need help with a billing dispute",
    "retrieved_chunks": ["For billing disputes...", ...],
    "confidence": 0.31,
    "response": "[Human Agent] I've escalated your billing dispute to...",
    "escalate": True,
    "human_response": "I've escalated your billing dispute to..."
}
```

---

3. LangGraph Workflow Design

3.1 Node Definitions

Node	Function	Reads From State	Writes To State	Responsibility

`retrieve_node`	`retrieve_node()`	`query`	`retrieved_chunks`, `confidence`	Load vector store, embed query, perform similarity search, compute average confidence
`generate_node`	`generate_node()`	`query`, `retrieved_chunks`	`response`	Construct prompt, invoke LLM, parse response string
`hitl_node`	`hitl_node()`	`query`, `response`, `confidence`	`escalate`, `human_response`, `response`	Present context to human, collect response, override LLM response

3.2 Edge Definitions

From	To	Type	Condition
`START`	`retrieve_node`	Entry point	Always
`retrieve_node`	`generate_node`	Direct edge	Always
`generate_node`	`hitl_node`	Conditional edge	`should_escalate()` returns `True`
`generate_node`	`END`	Conditional edge	`should_escalate()` returns `False`
`hitl_node`	`END`	Direct edge	Always

3.3 State Transitions Diagram

```
            ┌──────────┐
            │  START   │
            └────┬─────┘
                 │
            ┌────▼──────────┐
            │ retrieve_node │
            │               │
            │ state += {    │
            │   chunks,     │
            │   confidence  │
            │ }             │
            └────┬──────────┘
                 │ (always)
            ┌────▼──────────┐
            │ generate_node │
            │               │
            │ state += {    │
            │   response    │
            │ }             │
            └────┬──────────┘
                 │
            ┌────▼──────────────┐
            │ route_after_      │
            │ generate()        │
            └──┬────────────┬───┘
confidence    │            │  confidence >= 0.4
< 0.4 OR      │            │  AND no escalation
"escalate"    │            │  signals
in response   │            │
         ┌────▼─────┐  ┌──▼───┐
         │ hitl_node │  │ END  │
         │           │  └──────┘
         │ state += {│
         │  escalate,│
         │  human_   │
         │  response,│
         │  response │
         │ }         │
         └────┬──────┘
              │ (always)
         ┌────▼┐
```

```
                    │  END  │
                    └───────┘
```

3.4 State Passing Between Nodes

LangGraph manages state as a single dictionary that is passed through the graph. Each node function receives the full current state as input and returns a **partial dictionary** containing only the fields it modifies. LangGraph merges the returned dict into the current state using shallow update semantics (`state.update(node_output)`).

This means:

- Nodes do not need to return fields they did not change
- Nodes cannot accidentally delete fields set by earlier nodes
- The state grows monotonically as it passes through the graph

Example flow:

```
# After retrieve_node:
state = {"query": "...", "retrieved_chunks": [...], "confidence": 0.72,
         "response": "", "escalate": False, "human_response": None}

# After generate_node:
state = {"query": "...", "retrieved_chunks": [...], "confidence": 0.72,
         "response": "Our return policy...", "escalate": False, "human_response":
None}

# After hitl_node (if triggered):
state = {"query": "...", "retrieved_chunks": [...], "confidence": 0.31,
         "response": "[Human Agent] ...", "escalate": True,
         "human_response": "I've escalated..."}
```

---

4. Conditional Routing Logic

4.1 Routing Decision Table

#	Condition	Route	Reason
1	`confidence < 0.4`	`hitl_node`	Retrieval quality too low — context likely irrelevant; LLM would hallucinate
2	`"escalate"` in `response.lower()`	`hitl_node`	LLM explicitly signals it cannot answer from provided context

3	`"i don't know"` in `response.lower()`	`hitl_node`	LLM expresses uncertainty — response is not trustworthy
4	`"not sure"` in `response.lower()`	`hitl_node`	Same as above — alternative uncertainty phrasing
5	`"cannot answer"` in `response.lower()`	`hitl_node`	Same as above — another uncertainty variant
6	`"no relevant information"` in `response.lower()`	`hitl_node`	LLM reports insufficient context
7	None of the above	`END`	Confident, grounded response — safe to return directly

4.2 Confidence Scoring Heuristic

```
confidence = sum(scores) / len(scores) if scores else 0.0
```

The confidence score is the arithmetic mean of the top-k relevance scores returned by ChromaDB. Relevance scores are in [0, 1] where 1 = exact match and 0 = completely dissimilar.

Why average, not max or min?

- **Max** would inflate confidence when one good chunk is retrieved alongside irrelevant ones
- **Min** would be overly conservative — a single outlier chunk would trigger unnecessary escalation
- **Average** provides a balanced signal that reflects overall retrieval quality across all top-k results

4.3 Escalation Criteria Summary

The system escalates when any one of these conditions is true (OR logic):

1. **Low retrieval confidence** (`< 0.4`): The vector store did not find relevant content for this query. This is a data coverage gap.
2. **LLM self-assessment**: The LLM, despite receiving context, determines it cannot provide a reliable answer. The system prompt instructs it to include "ESCALATE" in such cases.
3. **Topic-based escalation**: The system prompt explicitly tells the LLM to escalate billing disputes, account security issues, and legal matters — topics that require human judgment regardless of context quality.

---

5. HITL Design

5.1 Trigger Conditions

HITL is triggered by `should_escalate(confidence, response)` in `src/hitl.py`. This function is called by `route_after_generate()` in `src/graph.py`. The six specific trigger conditions are listed in the

Routing Decision Table above.

5.2 Data Passed to Human

The human operator receives:

Data Field	Source	Purpose
`query`	`state["query"]`	The original customer question — what needs answering
`llm_response`	`state["response"]`	The LLM's attempted answer — may contain useful partial information
`confidence`	`state["confidence"]`	Retrieval quality score — helps the human assess why escalation occurred

This triad gives the human operator enough context to provide an informed answer without needing to re-read source documents.

5.3 Human Response Capture

```
human_response = input("\n✍  Your response: ").strip()
```

The function blocks on `input()` until the human types a response and presses Enter. Empty responses are replaced with a default: `"A human agent has reviewed this query and will follow up shortly."`

5.4 Re-entry into Graph State

The `hitl_node` returns a partial state update:

```
return {
    "escalate": True,
    "human_response": human_response,
    "response": f"[Human Agent] {human_response}"
}
```

This overwrites `state["response"]` with the human's answer, prefixed with `[Human Agent]` for clear attribution. The original LLM response is effectively discarded.

5.5 Limitations of Current CLI Implementation

1. **Blocking**: The `input()` call blocks the entire process. In a multi-user system, this would halt all query processing.
2. **Single operator**: Only one human can respond at a time. There is no operator routing, queue management, or workload distribution.

3. **No timeout**: If the human walks away, the system hangs indefinitely.
4. **No audit trail**: Human responses are not persisted beyond the current session.
5. **No feedback loop**: The human's response is not used to improve future retrieval or generation.

5.6 Production HITL Architecture

In production, the HITL module would be replaced with an async queue-based system:

```
hitl_node → POST to Webhook (Slack/Teams/PagerDuty)
          → Create ticket in queue (SQS/Redis/Celery)
          → Return interim response: "Your query has been escalated.
             A support agent will respond within 2 hours."
          → Agent picks up ticket from queue
          → Agent responds via admin UI
          → Callback/webhook triggers state update
          → Final response delivered to customer (email/chat)
```

This decouples the escalation trigger from the response collection, eliminating the blocking problem and enabling SLA-based response management.

---

6. API / Interface Design

6.1 Input Format (JSON Schema)

```
{
    "type": "object",
    "required": ["query"],
    "properties": {
        "query": {
            "type": "string",
            "description": "Customer's natural language question",
            "minLength": 1,
            "maxLength": 1000
        }
    }
}
```

6.2 Output Format (JSON Schema)

```
{
    "type": "object",
    "properties": {
        "query": {"type": "string"},
        "response": {"type": "string"},
        "confidence": {"type": "number", "minimum": 0, "maximum": 1},
        "escalated": {"type": "boolean"},
        "human_response": {"type": ["string", "null"]},
        "retrieved_chunks": {
            "type": "array",
            "items": {"type": "string"}
        }
    }
}
```

6.3 Interaction Flow

```
CLI Mode:
  User types query → run_query(query) → print result

API Mode (future):
  POST /query {"query": "..."} → 200 {"response": "...", "confidence": 0.72, ...}
  POST /ingest {"path": "..."} → 200 {"chunks_stored": 42}
  GET  /health                 → 200 {"status": "ok", "collection_size": 42}
```

---

7. Error Handling Matrix

#	Error Type	Detection	Handling Strategy
1	PDF file not found	`os.path.exists()` check in `load_pdf()`	Raise `FileNotFoundError` with descriptive message
2	No PDFs in directory	`glob.glob()` returns empty list	Raise `ValueError` with directory path
3	Empty PDF (no text)	`PyPDFLoader.load()` returns empty list	Log warning; skip file; continue with remaining PDFs
4	Vector store not initialized	`os.path.exists(CHROMA_PERSIST_DIR)` check	Raise `FileNotFoundError` instructing user to run ingestion

5	Empty vector store	`collection.count() == 0`	Log warning; retrieval returns empty results; confidence = 0.0; triggers HITL
6	LLM API key missing	API call raises `AuthenticationError`	Exception propagates with clear error message; user checks `.env`
7	LLM API rate limit	API returns 429 status	Exception propagates; user retries after backoff
8	LLM API timeout	API call exceeds timeout	Exception propagates; user retries
9	Embedding model download fails	`HuggingFaceEmbeddings` raises connection error	Exception propagates; user checks internet connection
10	Invalid query (empty string)	`input().strip()` returns empty string	`handle_interactive()` skips with `continue`
11	Human provides empty HITL response	`input().strip()` returns empty string	Default message: "A human agent has reviewed this query..."
12	ChromaDB corruption	ChromaDB raises internal error on load	Delete `chroma_db/` directory and re-run ingestion

---

Document version 1.0 — April 2026